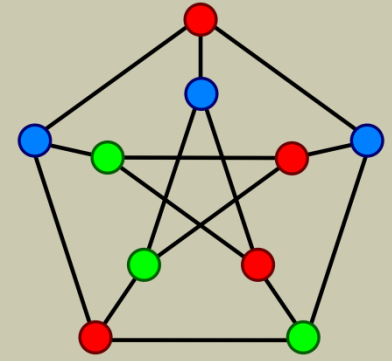
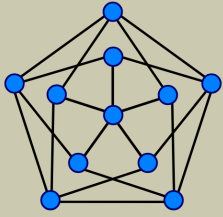
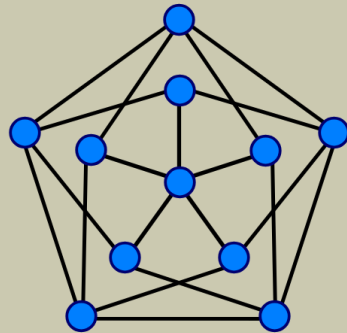
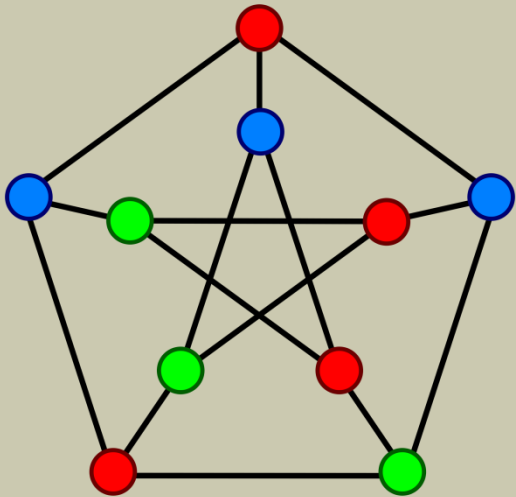
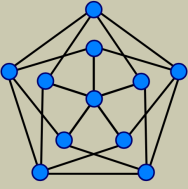




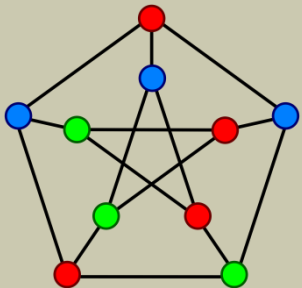
KOLOROWANIE GRAFÓW

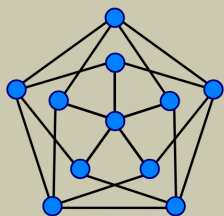


WSTĘP TEORETYCZNY



- Kolorowanie grafu to przypisanie kolorów wierzchołkom (lub krawędziom) grafu w taki sposób, aby żadne dwa sąsiednie wierzchołki nie miały tego samego koloru.
- Minimalna liczba kolorów potrzebna do poprawnego kolorowania to liczba chromatyczna grafu.
- Problem kolorowania grafów jest klasyczny w teorii grafów i należy do problemów bardzo trudnych.
- Ma zastosowanie w praktyce m.in. w:
 - Planowanie egzaminów lub zajęć – aby nie nakładały się w czasie.
 - Kolorowanie map – różne regiony muszą mieć inne kolory.
 - Przydzielanie częstotliwości radiowych – unikanie zakłóceń między nadajnikami.
 - Analiza konfliktów i zależności w grafach.
- Rozwiązania mogą wykorzystywać algorytmy zachłanne i dokładne.





RODZAJE ANALIZOWANYCH ALGORYTMÓW

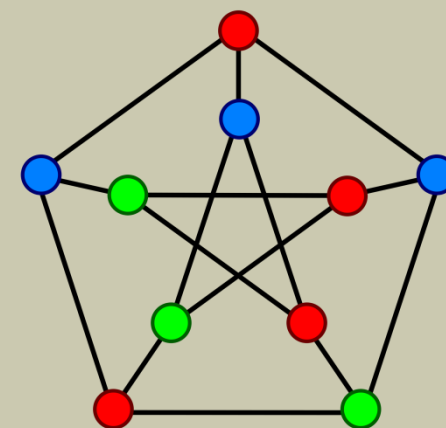
W projekcie zaimplementowano i porównano 5 algorytmów:

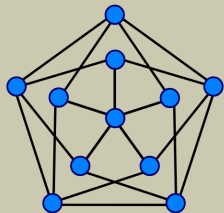
Zachłanne:

- Largest First (LF) – sortowanie według malejącego stopnia wierzchołka
- Smallest Last (SL) – Kolorowanie zaczynając od wierzchołków z najmniejszą liczbą połączeń.
- Saturation Largest First (SLF / DSATUR) – wybór na podstawie liczby różnych kolorów sąsiadów

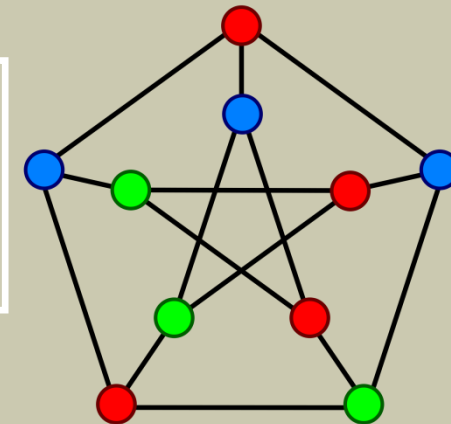
Dokładne:

- Brute Force – przegląd wszystkich możliwych kolorowań
- Backtracking – rekurencyjne przeszukiwanie z możliwością cofania





PROBLEM KOŁOROWANIA GRAFÓW

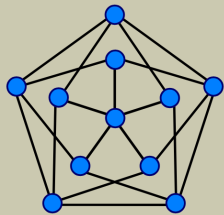


Wierzchołki reprezentują obiekty, które należy pokolorować, a krawędzie wskazują zależności konfliktowe – czyli przypadki, w których dwa elementy nie mogą mieć tego samego koloru.

W naszym projekcie skupiliśmy się na grafach losowych, ponieważ:

- pozwalają modelować różnorodne i nieprzewidywalne struktury danych,
- umożliwiają badanie zachowania algorytmów w typowych, a nie tylko ekstremalnych przypadkach,
- dają dużą swobodę w sterowaniu parametrami, takimi jak liczba wierzchołków czy gęstość krawędzi.

Dzięki temu mogliśmy przeanalizować skuteczność algorytmów kolorujących w wielu różnych konfiguracjach.



LARGEST FIRST (LF)

Opis:

Ten algorytm rozpoczyna kolorowanie od wierzchołków najbardziej "ważnych" – czyli tych o największym stopniu (liczbie sąsiadów).

Kroki:

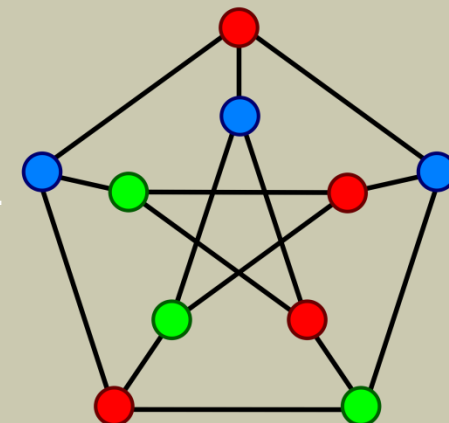
1. Oblicz stopień (liczbę krawędzi) każdego wierzchołka.
2. Posortuj wierzchołki malejąco według stopnia.
3. Przejdź po wierzchołkach w tej kolejności:
 1. Dla każdego wierzchołka przypisz najmniejszy możliwy kolor, który nie występuje u jego sąsiadów.

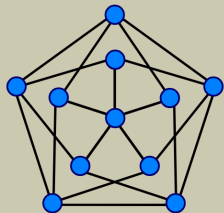
Zalety:

- Szybki i prosty w implementacji.
- Działa dobrze, gdy wierzchołki o dużym stopniu mają kluczowe znaczenie w strukturze grafu.

Wady:

- Nie zawsze daje optymalne rozwiązanie.
- Wrażliwy na strukturę grafu.





SMALLEST LAST (SL)

Opis:

Algorytm wybiera do kolorowania wierzchołki w odwrotnej kolejności usuwania: najpierw usuwa te o najmniejszym stopniu, a potem koloruje od końca.

Kroki:

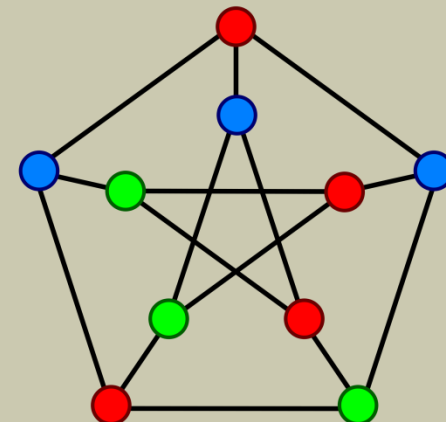
1. Iteracyjnie usuwaj wierzchołki o najmniejszym bieżącym stopniu, zapamiętując kolejność usunięcia.
2. Odwróć kolejność uzyskaną w kroku 1.
3. Przejdź przez wierzchołki w tej kolejności i koloruj je najniższym możliwym kolorem.

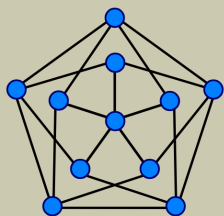
Zalety:

- Często lepsze niż LF w grafach gęstych lub nierównomiernych.
- Stara się unikać złych decyzji na początku kolorowania.

Wady:

- Nadal może używać więcej kolorów niż potrzeba.





SATURATION LARGEST FIRST (SLF / DSATUR)

Opis:

Najbardziej zaawansowany z algorytmów zachłannych. Wybiera wierzchołek o największym **saturacyjnym stopniu** (czyli liczbie różnych kolorów sąsiadów).

Kroki:

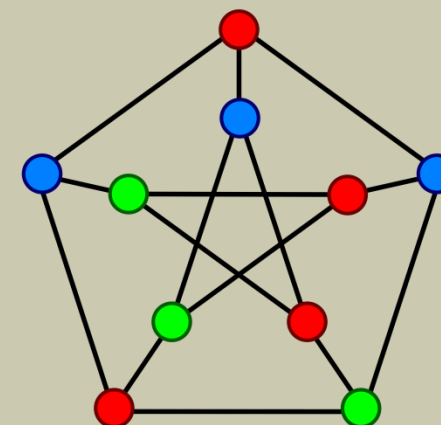
1. Tworzymy kopię grafu.
 2. Iteracyjnie:
 - wybieramy wierzchołek o najmniejszym stopniu,
 - usuwamy go z grafu,
 - dodajemy go na koniec listy kolejności.
- Robimy tak, aż graf będzie pusty.

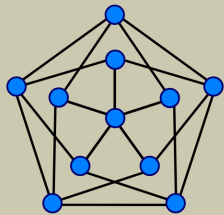
Zalety:

- Bardzo dobra jakość kolorowania.
- Używa mniejszej liczby kolorów niż inne heurystyki.
- Dobrze radzi sobie z trudnymi przypadkami.

Wady:

- Wolniejszy niż LF i SL (ale nadal dużo szybszy niż algorytmy dokładne).





BRUTE FORCE

Opis:

Algorytm sprawdza **wszystkie możliwe kombinacje kolorów** dla wierzchołków i wybiera tę, która używa najmniejszej liczby kolorów bez konfliktów.

Kroki:

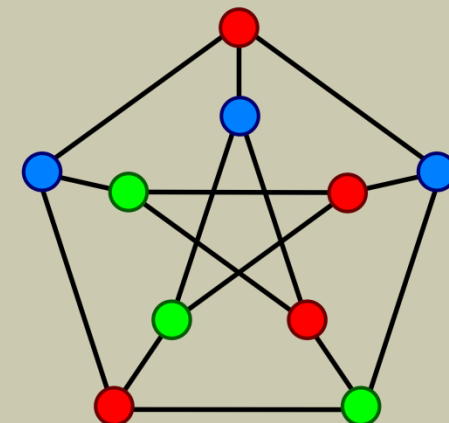
1. Wygeneruj wszystkie możliwe kombinacje kolorowań dla ustalonej liczby kolorów (np. od 1 do n).
2. Dla każdej kombinacji sprawdź, czy jest poprawna (żaden sąsiad nie ma tego samego koloru).
3. Wybierz ten z najmniejszą liczbą kolorów.

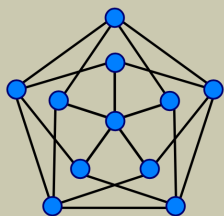
Zalety:

- Gwarantuje optymalne kolorowanie.

Wady:

- Złożoność: wykładnicza – $O(k^n)$ dla k kolorów i n wierzchołków.
- Niewykonalny dla większych grafów ($n > 16$).





BACKTRACKING

Opis:

Rekurencyjna wersja Brute Force z optymalizacją. Gdy znajdzie konflikt, **cofa się** i próbuje inny kolor – dzięki temu przeszukuje tylko część przestrzeni rozwiązań.

Kroki:

1. Dla każdego wierzchołka:

1. Przypisz kolor, jeśli jest bezpieczny (nie ma konfliktu z sąsiadami).
2. Przejdź do kolejnego wierzchołka.

2. Jeśli nie ma możliwości – cofnij się i spróbuj inny kolor.

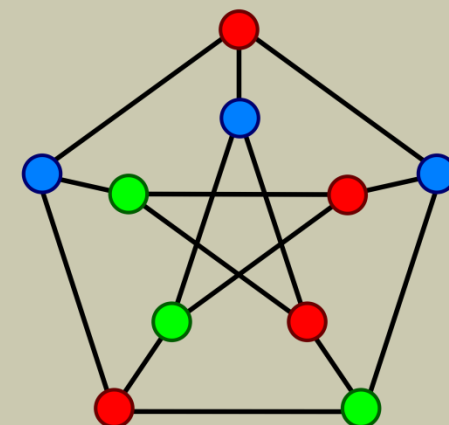
3. Jeśli kolorowanie zakończy się sukcesem – zapisz wynik.

Zalety:

- Znacznie szybszy niż Brute Force dzięki eliminowaniu błędnych ścieżek.
- Nadaje się dla średnich grafów.

Wady:

- Nadal wykładnicza złożoność w najgorszym przypadku.



DZIAŁANIE PĘTLI ALGORYTMÓW ZACHŁANNYCH

PĘTLA ALGORYTMU LF :

- Sortuje wierzchołki malejąco według liczby sąsiadów.
- Dla każdej wierzchołka (for w) wybiera **najmniejszy dostępny kolor**.
- Wewnętrzna pętla (while) sprawdza, czy kolor jest już zajęty przez sąsiadów.

Działa bardzo szybko – tylko lokalnie sprawdza sąsiedztwo.

```
def zachlanny_ogolny(graf: nx.Graph, kolejnosc: list) -> dict:
    kolorowanie = {}
    for w in kolejnosc:
        zajete = {kolorowanie[s] for s in graf.neighbors(w) if s in kolorowanie}
        kolor = 0
        while kolor in zajete:
            kolor += 1
        kolorowanie[w] = kolor
    return kolorowanie

def zachlanny_lf(graf: nx.Graph) -> dict:
    return zachlanny_ogolny(graf, sorted(graf.nodes(), key=lambda x: graf.degree[x], reverse=True))
```

PĘTLA ALGORYTMU SL :

- Pętla while usuwa iteracyjnie wierzchołki o najmniejszym stopniu.
- Tworzy listę, która będzie kolorowana **odwrotnie**.
- Dalsze kolorowanie jak w LF – ta sama zachlanny_ogolny.

To tak zwana **Degeneracja grafu**. Działa lepiej niż LF, bo zaczyna od najłatwiejszych wierzchołków.

```
def zachlanny_sl(graf: nx.Graph) -> dict:
    tymczasowy = graf.copy()
    kolejnosc = []
    while tymczasowy.nodes:
        w = min(tymczasowy.nodes, key=lambda x: tymczasowy.degree[x])
        kolejnosc.append(w)
        tymczasowy.remove_node(w)
    return zachlanny_ogolny(graf, reversed(kolejnosc))
```

DZIAŁANIE PĘTLI ALGORYTMÓW ZACHŁANNYCH

Pętla Algorytmu SLF :

- **Pętla (while) :**
 - działa, dopóki nie pokolorujesz wszystkich wierzchołków
- **W środku:**
 - wybiera **wierzchołek o największym nasyceniu**, czyli ma sąsiadów z największą liczbą różnych kolorów.
 - jeśli nasycenia są równe → bierze ten o największym stopniu
- **Druga pętla (for s in graf.neighbors(w)) :**
 - aktualizuje nasycenie sąsiadów, którzy jeszcze nie są pokolorowani

Jest to najbardziej „inteligentna” heurystyka – dynamicznie wybiera najlepszy w danym momencie wierzchołek.

```
def zachlanny_slf(graf: nx.Graph) -> dict:
    kolorowanie = {}
    nasycenie = defaultdict(int)
    stopnie = dict(graf.degree())

    while len(kolorowanie) < len(graf.nodes()):
        niepokolorowane = [n for n in graf.nodes if n not in kolorowanie]
        w = max(niepokolorowane, key=lambda n: (nasycenie[n], stopnie[n]))

        zajete = {kolorowanie[s] for s in graf.neighbors(w) if s in kolorowanie}
        kolor = 0
        while kolor in zajete:
            kolor += 1
        kolorowanie[w] = kolor

        for s in graf.neighbors(w):
            if s not in kolorowanie:
                nasycenie[s] = len({kolorowanie[n] for n in graf.neighbors(s) if n in kolorowanie})

    return kolorowanie
```

DZIAŁANIE PĘTLI ALGORYTMÓW DOKŁADNYCH

```
def waliduj_kolorowanie(graf, kolorowanie):  
    return all(kolorowanie[u] != kolorowanie[v] for u, v in graf.edges())  
  
def brutalne_kolorowanie(graf: nx.Graph, maks_kolorow=None):  
    wierzcholki = list(graf.nodes())  
    n = len(wierzcholki)  
    maks_kolorow = maks_kolorow or n  
  
    for k in range(1, maks_kolorow + 1):  
        for kombinacja in itertools.product(range(k), repeat=n):  
            przypisanie = dict(zip(wierzcholki, kombinacja))  
            if waliduj_kolorowanie(graf, przypisanie):  
                return przypisanie  
    return {}
```

ALGORYTM BRUTE FORCE :

- **Pierwsza pętla (for k):**
 - Próbuje rosnące liczby kolorów: 1, 2, 3, ..., n.
 - Celem jest znalezienie minimalnej liczby kolorów, która wystarczy.
- **Druga pętla (for kombinacja):**
 - Generuje wszystkie możliwe kombinacje przypisań kolorów do wierzchołków.
 - Dla 3 wierzchołków i 2 kolorów: (0,0,0), (0,0,1), (0,1,0), i tak dalej.
 - Testuje każde przypisanie, aż znajdzie poprawne (takie, że sąsiedzi mają różne kolory).

To pętla brutalna sprawdza wszystkie możliwe opcje, dlatego działa długo.

DZIAŁANIE PĘTLI ALGORYTMÓW DOKŁADNYCH

```
def kolorowanie_wsteczne(graf: nx.Graph) -> dict:
    wierzcholki = list(graf.nodes())
    przypisanie = {}

    def bezpieczne(w, kolor):
        return all(przypisanie.get(s) != kolor for s in graf.neighbors(w))

    def rekurencja(index, maks):
        if index == len(wierzcholki):
            return True
        w = wierzcholki[index]
        for k in range(maks):
            if bezpieczne(w, k):
                przypisanie[w] = k
                if rekurencja(index + 1, maks):
                    return True
                del przypisanie[w]
        return False

    for k in range(1, len(wierzcholki) + 1):
        przypisanie.clear()
        if rekurencja(0, k):
            return przypisanie
    return {}
```

ALGORYTM BACKTRACKING :

Główna pętla:

- Przechodzi przez rosnące liczby kolorów, tak jak Brute Force, ale tylko **jeden raz dla każdego k**.
- Jeśli znajdzie poprawne przypisanie, kończy od razu.

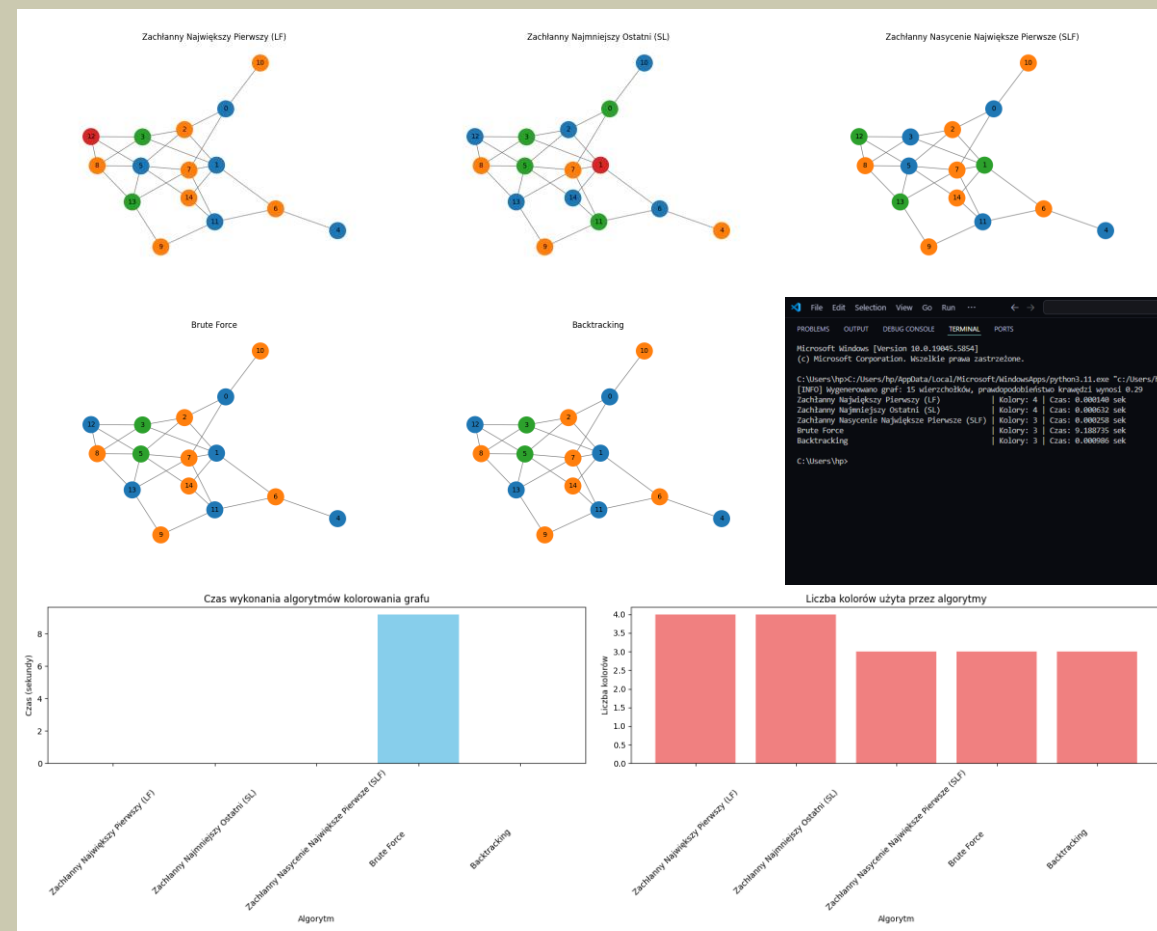
Rekurencyjna pętla ukryta w funkcji (rekurencja(...)) :

- Przechodzi przez możliwe kolory (od 0 do maks - 1) .
- Sprawdza, czy można przypisać kolor bez konfliktu z sąsiadami.
- Jeśli się nie uda → **cofa się (Backtrack)** i próbuje inne przypisanie.

Nie generuje wszystkich kombinacji, tylko buduje rozwiązanie krok po kroku i wycofuje się, jeśli trafia w ślepy zaułek.

WYNIKI PROGRAMU KOLORUJĄCEGO GRAFY

Po uruchomieniu programu, każdy z algorytmów kolorowania grafu jest wykonywany na tym samym losowo wygenerowanym grafie. Program mierzy czas działania każdego algorytmu oraz liczy, ile kolorów zostało użytych do pokolorowania grafu. Wyniki są prezentowane w formie czytelnej tabeli w terminalu, dzięki czemu można je łatwo porównać. Dodatkowo program generuje dwa wykresy – jeden przedstawia czas działania algorytmów, a drugi pokazuje liczbę kolorów użytych przez każdy z nich. Oprócz tego, dla każdego algorytmu wyświetlana jest wizualizacja grafu z nałożonym kolorowaniem, co umożliwia graficzne porównanie wyników. Jeśli graf ma zbyt dużo wierzchołków, niektóre algorytmy, na przykład Brute Force, są pomijane, aby uniknąć długiego czasu obliczeń.



ANALIZA PROBLEMU KOLOWANIA GRAFÓW PRZY POMOCY ALGORYTMÓW ZACHŁANNYCH

Na potrzeby projektu i analizy problemu kolorowania grafów wybraliśmy losowy graf o 10 wierzchołkach oraz prawdopodobieństwie krawędzi wynoszącym 0.4. Kolorowanie przeprowadzono za pomocą trzech algorytmów zachłannych:

- **LF (Largest First),**
- **SL (Smallest Last).**
- **SLF (Saturation Largest First / DSATUR).**

Dla każdego algorytmu zachłannego wypisywano:

- aktualnie kolorowany wierzchołek,
- jego sąsiadów,
- kolory już zajęte przez sąsiadów,
- kolor przypisany do danego wierzchołka.

Każdy z algorytmów przyjął inną strategię ustalania kolejności kolorowania, co wpłynęło na rozkład kolorów w grafie. Pomimo tego, wszystkie działały zgodnie z zasadą zachłanności – przypisywały wierzchołkom pierwszy możliwy kolor, który nie występował jeszcze w ich bezpośrednim sąsiedztwie. Prezentowane wyniki pozwalają na dokładne prześledzenie działania poszczególnych metod i porównanie ich efektywności oraz różnic w podejściu do problemu kolorowania grafu.

```
[INFO] Wygenerowano graf: 10 wierzchołków, prawdopodobieństwo krawędzi wynosi 0.4
```

```
[INFO] Start: Zachłanny Największy Pierwszy (LF)
```

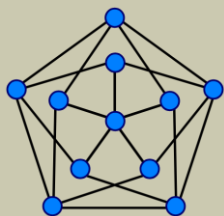
```
Wierzchołek [2] -> kolor: 0, sąsiedzi: [1, 4, 7, 8], kolory sąsiadów: set()
Wierzchołek [3] -> kolor: 0, sąsiedzi: [1, 4, 5, 9], kolory sąsiadów: set()
Wierzchołek [4] -> kolor: 1, sąsiedzi: [2, 3, 6, 7], kolory sąsiadów: {0}
Wierzchołek [7] -> kolor: 2, sąsiedzi: [0, 2, 4, 5], kolory sąsiadów: {0, 1}
Wierzchołek [0] -> kolor: 0, sąsiedzi: [6, 7, 9], kolory sąsiadów: {2}
Wierzchołek [1] -> kolor: 1, sąsiedzi: [2, 3, 5], kolory sąsiadów: {0}
Wierzchołek [5] -> kolor: 3, sąsiedzi: [1, 3, 7], kolory sąsiadów: {0, 1, 2}
Wierzchołek [6] -> kolor: 2, sąsiedzi: [0, 4, 8], kolory sąsiadów: {0, 1}
Wierzchołek [8] -> kolor: 1, sąsiedzi: [2, 6], kolory sąsiadów: {0, 2}
Wierzchołek [9] -> kolor: 1, sąsiedzi: [0, 3], kolory sąsiadów: {0}
```

```
[INFO] Start: Zachłanny Najmniejszy Ostatni (SL)
```

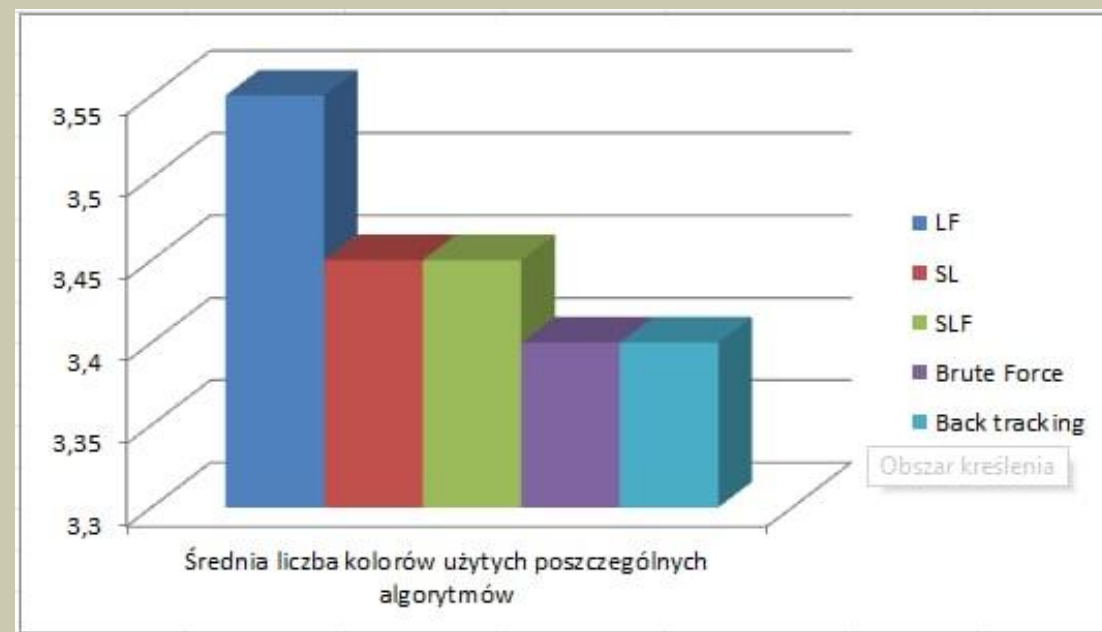
```
Wierzchołek [7] -> kolor: 0, sąsiedzi: [0, 2, 4, 5], kolory sąsiadów: set()
Wierzchołek [5] -> kolor: 1, sąsiedzi: [1, 3, 7], kolory sąsiadów: {0}
Wierzchołek [4] -> kolor: 1, sąsiedzi: [2, 3, 6, 7], kolory sąsiadów: {0}
Wierzchołek [3] -> kolor: 0, sąsiedzi: [1, 4, 5, 9], kolory sąsiadów: {1}
Wierzchołek [2] -> kolor: 2, sąsiedzi: [1, 4, 7, 8], kolory sąsiadów: {0, 1}
Wierzchołek [1] -> kolor: 3, sąsiedzi: [2, 3, 5], kolory sąsiadów: {0, 1, 2}
Wierzchołek [9] -> kolor: 1, sąsiedzi: [0, 3], kolory sąsiadów: {0}
Wierzchołek [0] -> kolor: 2, sąsiedzi: [6, 7, 9], kolory sąsiadów: {0, 1}
Wierzchołek [6] -> kolor: 0, sąsiedzi: [0, 4, 8], kolory sąsiadów: {1, 2}
Wierzchołek [8] -> kolor: 1, sąsiedzi: [2, 6], kolory sąsiadów: {0, 2}
```

```
[INFO] Start: Zachłanny Nasycenie Największe Pierwsze (SLF/DSATUR)
```

```
Wierzchołek [2] -> kolor: 0, stopień nasycenia: 0, kolory sąsiadów: set()
Wierzchołek [4] -> kolor: 1, stopień nasycenia: 1, kolory sąsiadów: {0}
Wierzchołek [7] -> kolor: 2, stopień nasycenia: 2, kolory sąsiadów: {0, 1}
Wierzchołek [3] -> kolor: 0, stopień nasycenia: 1, kolory sąsiadów: {1}
Wierzchołek [5] -> kolor: 1, stopień nasycenia: 2, kolory sąsiadów: {0, 2}
Wierzchołek [1] -> kolor: 2, stopień nasycenia: 2, kolory sąsiadów: {0, 1}
Wierzchołek [0] -> kolor: 0, stopień nasycenia: 1, kolory sąsiadów: {2}
Wierzchołek [6] -> kolor: 2, stopień nasycenia: 2, kolory sąsiadów: {0, 1}
Wierzchołek [8] -> kolor: 1, stopień nasycenia: 2, kolory sąsiadów: {0, 2}
Wierzchołek [9] -> kolor: 1, stopień nasycenia: 1, kolory sąsiadów: {0}
```

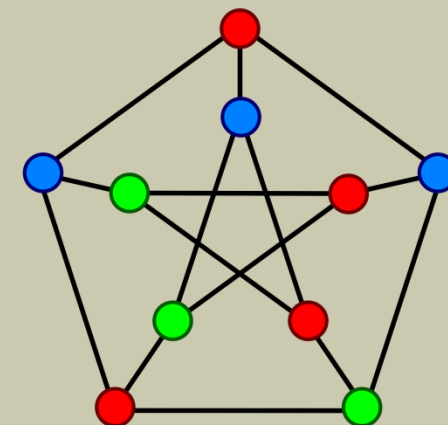



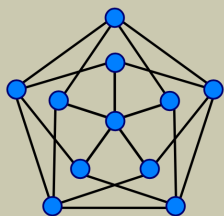
WYNIKI DLA GRAFÓW Z 12 WIERZCHOŁKAMI PRZY 20 PRÓBACH



Średnia czasu działania poszczególnego algorytmu				
LF	SL	SLF	Brute Force	Back tracking
0,000106885	0,00043637	0,00019475	2,79942585	0,002897965

Średnia liczba kolorów użytych poszczególnych algorytmów				
LF	SL	SLF	Brute Force	Back tracking
3,55	3,45	3,45	3,4	3,4





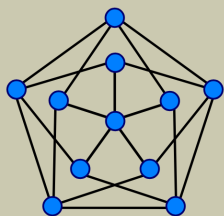
WNIOSKI DLA GRAFÓW Z 12 WIERZCHOŁKAMI PRZY 20 PRÓBACH

Efektywność czasowa:

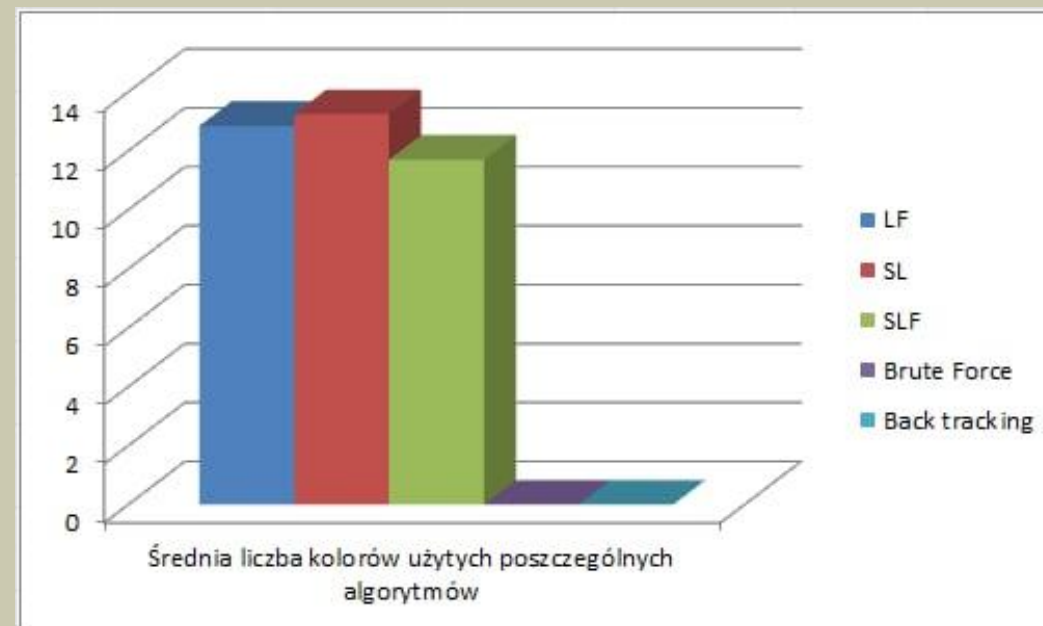
- Algorytm LF (Largest First) okazał się najszybszy (średni czas: $\sim 0,000107s$), co jest typowe dla prostych algorytmów zachłannych.
- Algorytm SL (Smallest Last) był około 4x wolniejszy od LF ($\sim 0,000436s$), ale nadal bardzo szybki.
- SLF zajmował pozycję pośrednią ($\sim 0,000195s$).
- Backtracking był znacząco wolniejszy ($\sim 0,0029s$), ale nadal praktyczny dla małych grafów.
- Brute Force był dramatycznie wolniejszy ($\sim 2,8s$), co potwierdza jego niepraktyczność nawet dla stosunkowo małych grafów (12 wierzchołków).

Liczba kolorów:

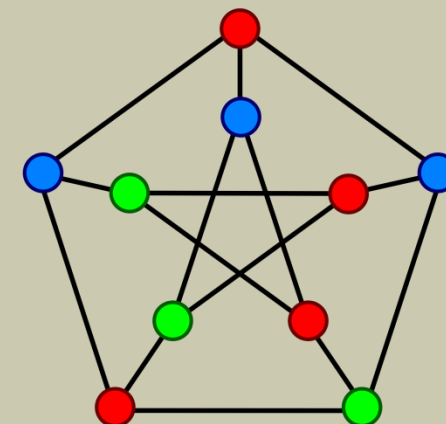
- Algorytmy Brute Force i Backtracking osiągnęły najlepsze wyniki (średnio 3,4 koloru), co było do przewidzenia, ponieważ są to algorytmy dokładne.
- Algorytmy heurystyczne (SL, SLF) osiągnęły bardzo zbliżone wyniki do dokładnych (3,45), co sugeruje ich wysoką skuteczność.
- Algorytm LF wypadł nieco gorzej (3,55), ale różnica jest stosunkowo niewielka.

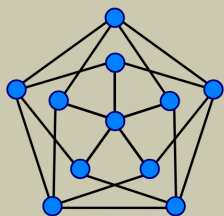


WYNIKI DLA GRAFÓW Z 100 WIERZCHOŁKAMI PRZY 20 PRÓBACH



Średnia czasu działania poszczególnego algorytmu				
LF	SL	SLF	Brute Force	Back tracking
0,001	0,010611945	0,010914055	N/D	N/D
Średnia liczba kolorów użytych poszczególnych algorytmów				
LF	SL	SLF	Brute Force	Back tracking
12,85	13,25	11,7	N/D	N/D





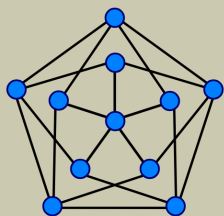
WNIOSKI DLA GRAFÓW Z 100 WIERZCHOŁKAMI PRZY 20 PRÓBACH

Efektywność czasowa:

1. Algorytmy Brute Force i Backtracking zostały oznaczone jako "N/D" (nieokreślone), co sugeruje, że ich czas wykonania dla grafów o 100 wierzchołkach był zbyt długi do praktycznego zastosowania lub nawet niemożliwy do zmierzenia w rozsądnym czasie.
2. Algorytmy heurystyczne (LF, SL, SLF) pozostały praktyczne nawet dla większych grafów, co potwierdza ich skalowalność.

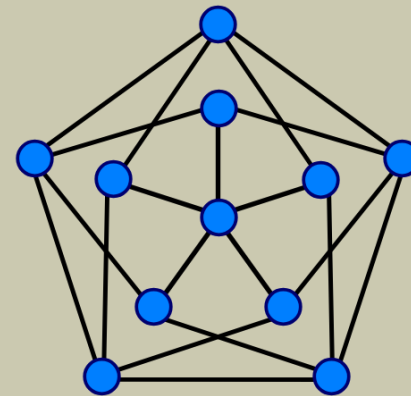
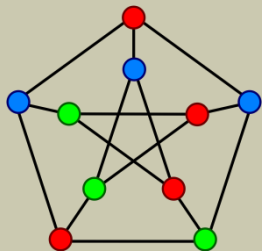
Liczba kolorów:

1. SLF osiągnął najlepszy wynik (11.7 kolorów), demonstrując swoją efektywność w minimalizacji liczby kolorów.
2. LF uzyskał średnio 12.85 kolorów, podczas gdy SL - 13.25 kolorów, co pokazuje, że kolejność przetwarzania wierzchołków ma znaczący wpływ na jakość kolorowania.
3. Brak wyników dla Brute Force i Backtracking.

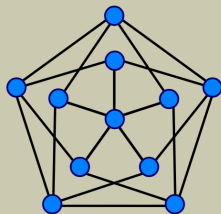
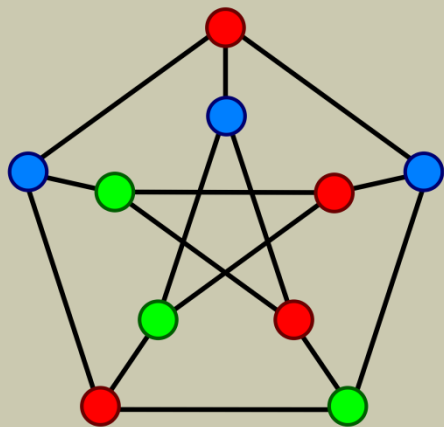


PODSUMOWANIE

- Algorytmy zachłanne - to najszybsze rozwiązania, doskonale radzą sobie z dużymi grafami ale nie gwarantują użycia minimalnej liczby kolorów.
- Algorytm Brute Force- zawsze znajduje optymalne rozwiązanie, jednak nie nadaje się do użycia już dla grafów większych niż 16 wierzchołków.
- Metoda Backtracking- znajduje rozwiązania porównywalne do metod zachłannych natomiast wymaga ograniczenia do grafów o 30 wierzchołkach.
- Algorytm SLF- oferuje lepszą jakość kolorowania niż inne metody zachłanne, jednak czas wykonania znacząco rośnie wraz z rozmiarem grafu



DZIĘKUJEMY ZA UWAGĘ



Wiktoria Nyklewicz 176842,
Kacper Lis 176828